Register Transfer Language and Microoperations

Unit-2

Contents:

1. Register Transfer Language

SIMPLE DIGITAL SYSTEMS

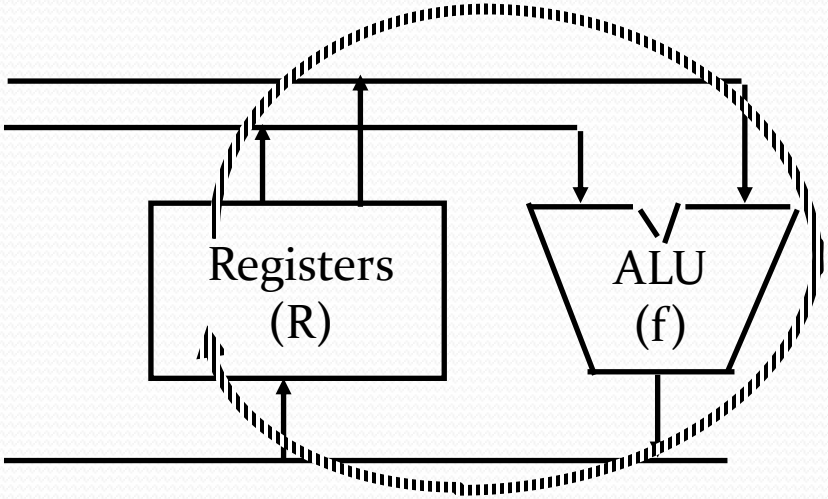
- Combinational and sequential circuits can be used to create simple digital systems.
- These are the low-level building blocks of a digital computer.
- Simple digital systems are frequently characterized in terms of
  - the registers they contain, and
  - the operations that they perform.
- Typically,
  - Operations are performed on the data in the registers.
  - Information is passed between registers.

Microoperations(1)

- The operations on the data in registers are called microoperations.
- The functions built into registers are examples of microoperations
  - Shift
  - Load
  - Clear
  - Increment
  - …

Microoperations (2)

An elementary operation performed (during one clock pulse), on the information stored in one or more registers.



1 clock cycle

$R \leftarrow f(R, R)$

f: shift, load, clear, increment, add, subtract, complement, and, or, xor, ...

Organization of a Digital System

The internal organization of a computer is defined as

1. Set of registers and their functions.
2. Microoperations (Set of allowable microoperations provided by the organization of the computer).
3. Control signals that initiate the sequence of microoperations (to perform the functions).

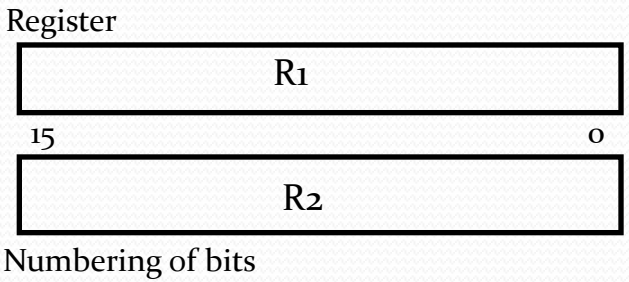
Register Transfer Level

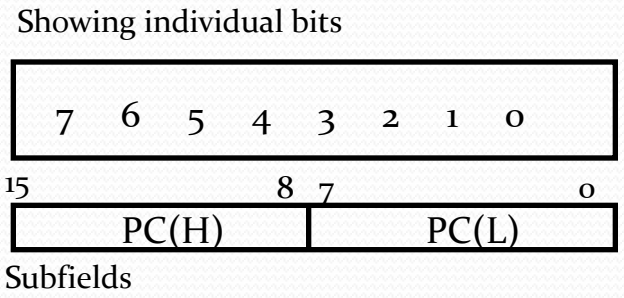
- Viewing a computer, or any digital system, in this way is called the register transfer level.
- This is because we're focusing on
  - The system's registers
  - The data transformations in them, and
  - The data transfers between them.

Register Transfer Language

- Rather than specifying a digital system in words, a specific notation is used called *register transfer language*.
- For any function of the computer, the register transfer language can be used to describe the (sequence of) microoperations.
- Register transfer language
  - A symbolic language.
  - A convenient tool for describing the internal organization of digital computers.
  - Can also be used to facilitate the design process of digital systems.

- Designation of a register
  - a register
  - portion of a register
  - a bit of a register
- Common ways of drawing the block diagram of a register





Basic Symbols For Register Transfers

Symbols	Description	Examples
Capital letters & numerals	Denotes a register	MAR, R2
Parentheses ()	Denotes a part of a register	R2(0-7), R2(L)
Arrow ←	Denotes transfer of information	R2 ← R1
Colon :	Denotes termination of control function	P:
Comma ,	Separates two micro-operations	A ← B, B ← A

2. Arithmetic Microoperations

- The microoperations most often encountered in digital computers are classified into four categories:
  - *Register transfer* microoperations
  - *Arithmetic* microoperations (on numeric data stored in the registers)
  - *Logic microoperations* (bit manipulations on non-numeric data)
  - *Shift microoperations*

Arithmetic Microoperations cont.

- The basic arithmetic microoperations are:
  - addition
  - subtraction
  - increment
  - decrement

- Addition Microoperation:

$$R3 \leftarrow R1+R2$$

- Subtraction Microoperation:

$$R3 \leftarrow R1-R2 \text{ (or)}$$

$$R3 \leftarrow R1+ \overline{R2} +1$$

1's complement

Arithmetic Microoperations cont

- One's Complement Microoperation:

$$R2 \leftarrow \overline{R2}$$

- Two's Complement Microoperation:

$$R2 \leftarrow \overline{R2+1}$$

- Increment Microoperation:

$$R2 \leftarrow R2+1$$

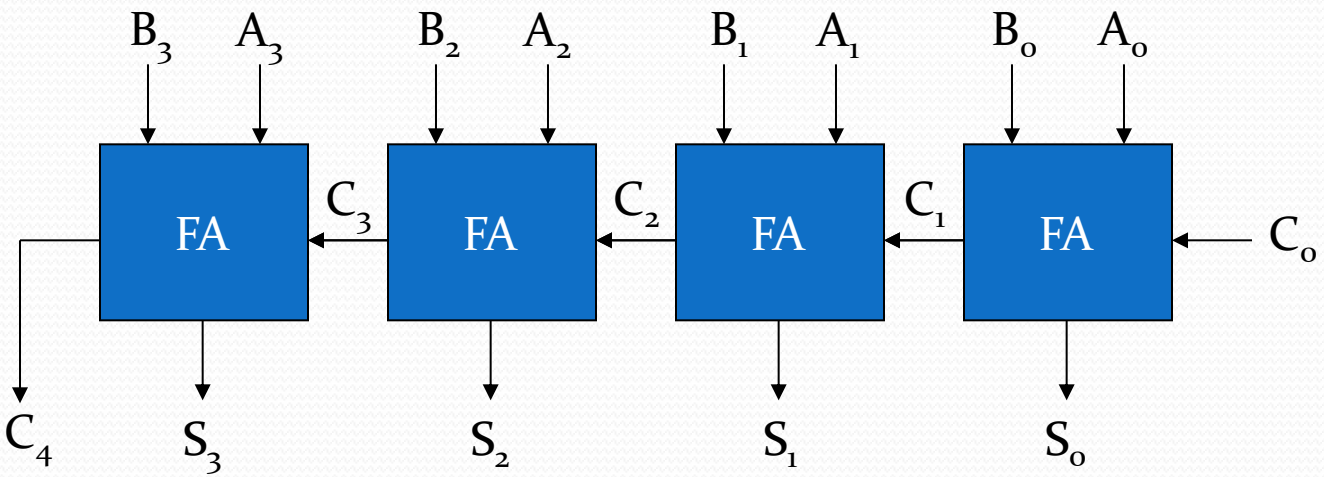
- Decrement Microoperation:

$$R2 \leftarrow R2-1$$

Summary of Typical Arithmetic Micro-Operations

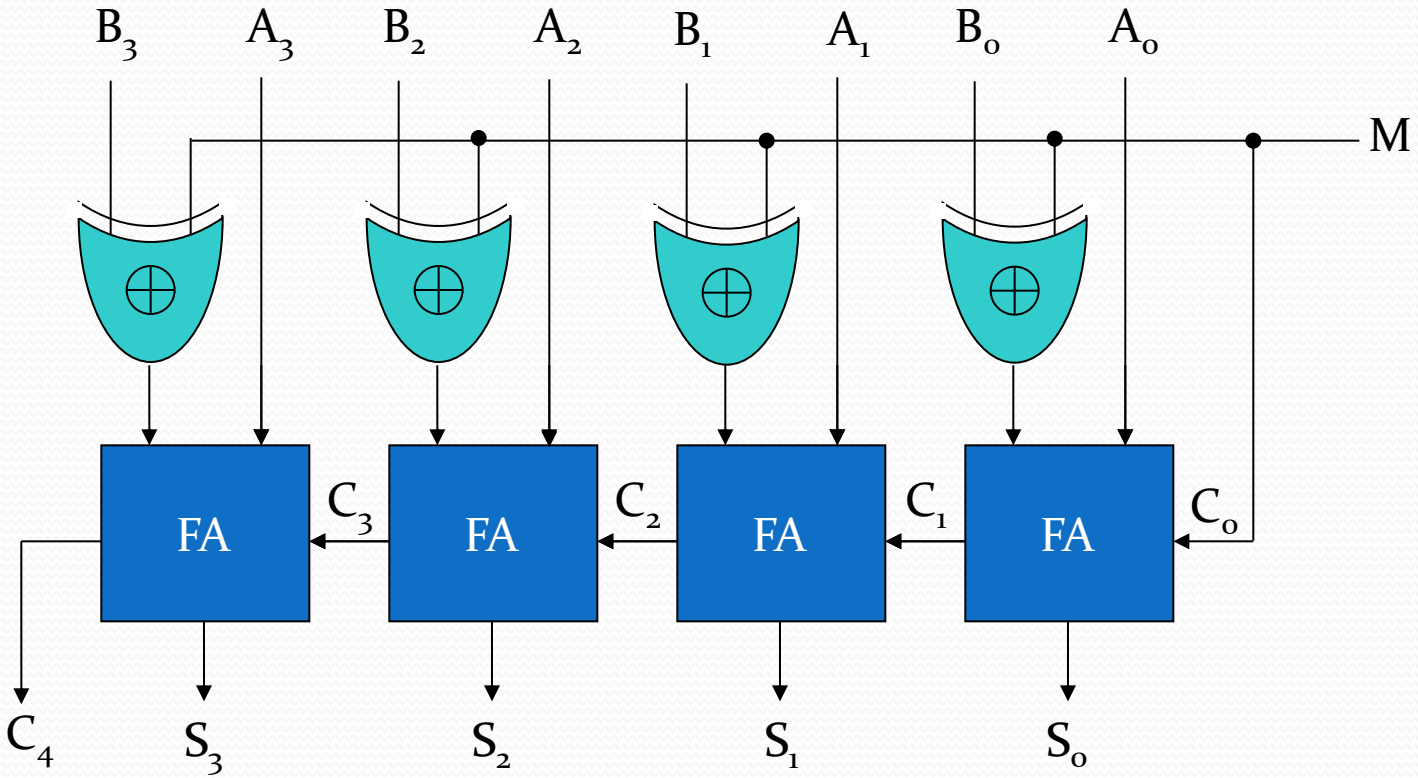
Operations	Description
$R3 \leftarrow R1 + R2$	Contents of R1 plus R2 transferred to R3
$R3 \leftarrow R1 - R2$	Contents of R1 minus R2 transferred to R3
$R2 \leftarrow \overline{R2}$	Complement the contents of R2
$R2 \leftarrow \overline{R2} + 1$	2's complement the contents of R2 (negate)
$R3 \leftarrow R1 + \overline{R2} + 1$	subtraction
$R1 \leftarrow R1 + 1$	Increment
$R1 \leftarrow R1 - 1$	Decrement

Arithmetic Microoperations: Binary Adder



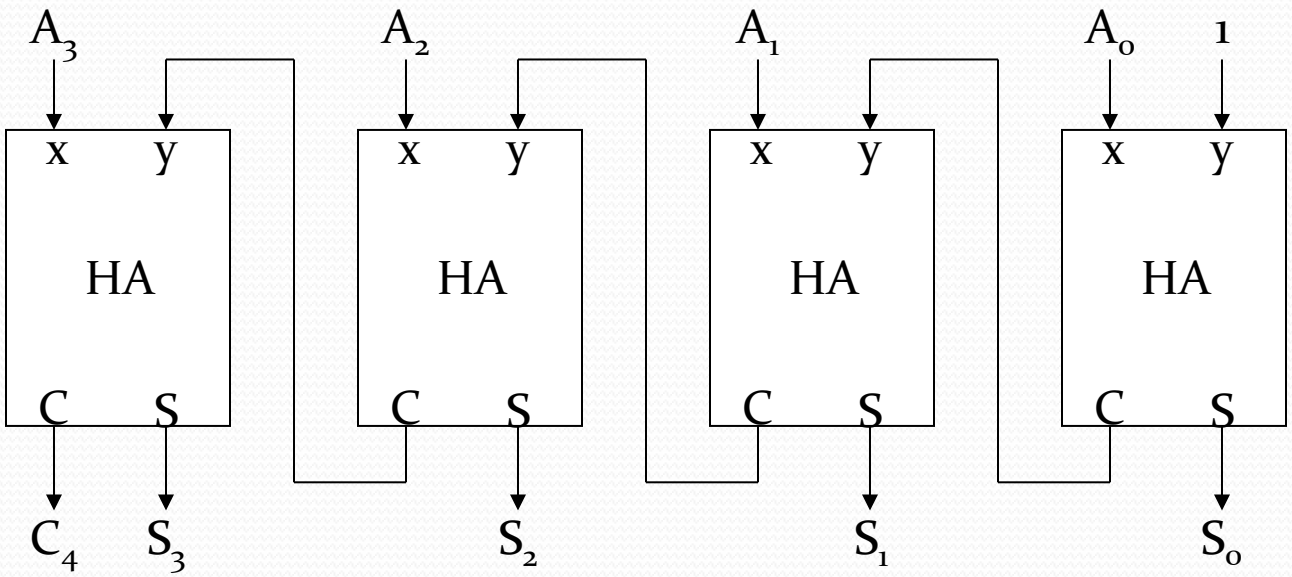
4-bit binary adder (connection of FAs)

Arithmetic Microoperations: Binary Adder-Subtractor



4-bit adder-subtractor

Arithmetic Microoperations Binary Incrementer



4-bit Binary Incrementer

- Binary Incrementer can also be implemented using a counter.
- A binary decrementer can be implemented by adding 1111 to the desired register each time.

Arithmetic Microoperations Arithmetic Circuit

- This circuit performs seven distinct arithmetic operations and the basic component of it is the parallel adder.
- The output of the binary adder is calculated from the following arithmetic sum:
  - $D = A + Y + C_{in}$

Arithmetic Microoperations : Arithmetic Circuit cont.

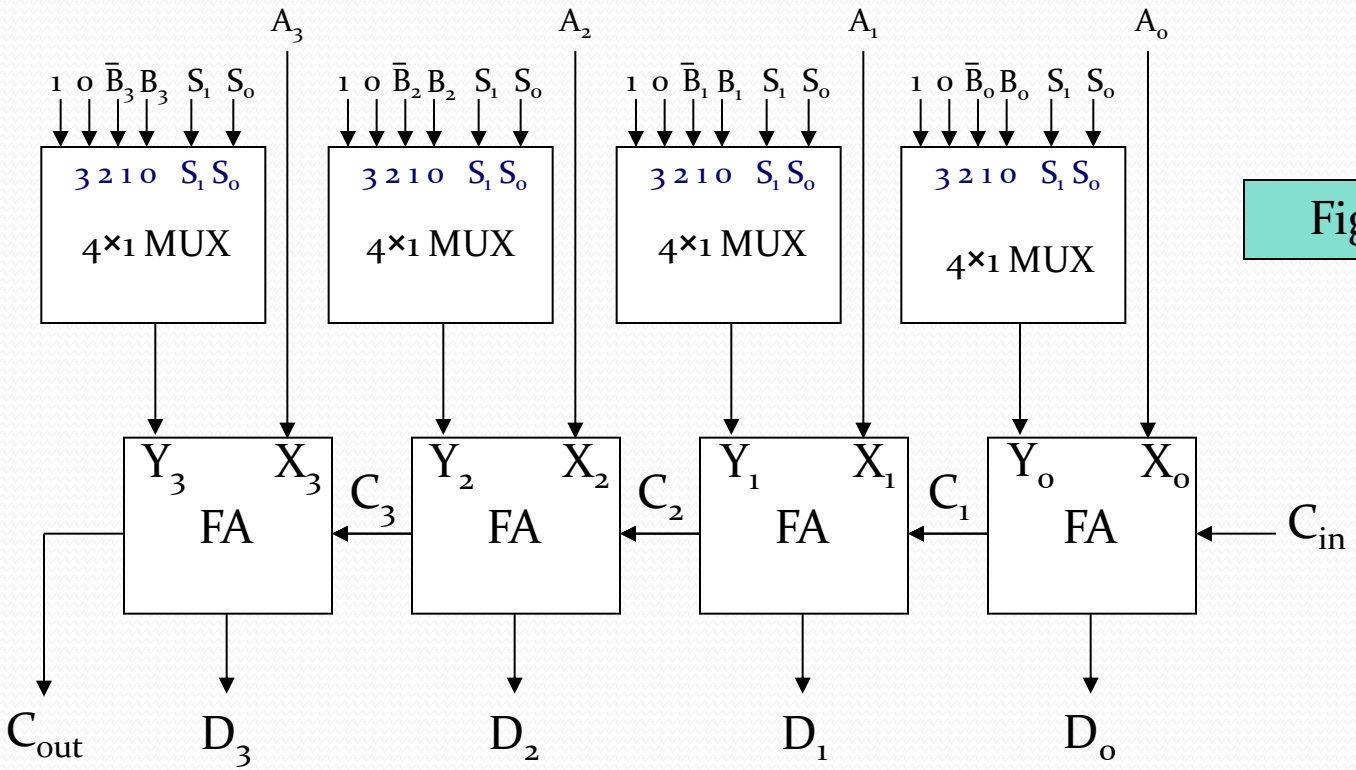
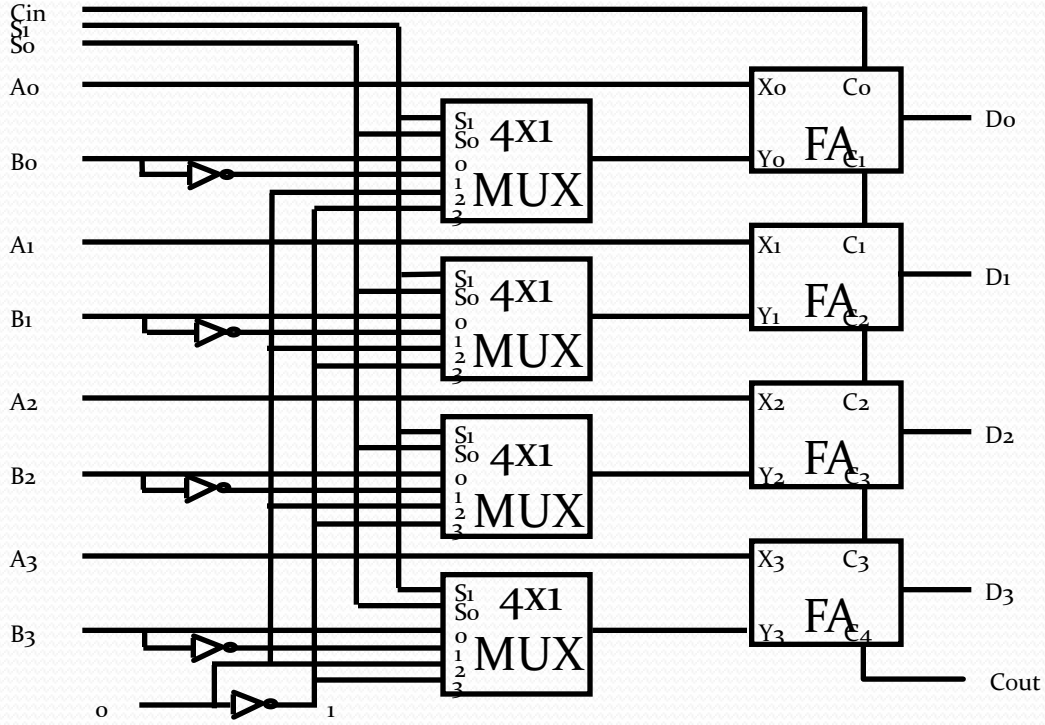


Figure A

4-bit Arithmetic Circuit

Arithmetic Circuit



S1	So	Cin	Y	Output	Microoperation
0	0	0	B	D = A + B	Add
0	0	1	B	D = A + B + 1	Add with carry
0	1	0	B'	D = A + B'	Subtract with borrow
0	1	1	B'	D = A + B'+ 1	Subtract
1	0	0	0	D = A	Transfer A
1	0	1	0	D = A + 1	Increment A
1	1	0	1	D = A - 1	Decrement A
1	1	1	1	D = A	Transfer A

3. Logic Microoperations

- Specify binary operations on the strings of bits in registers
  - Logic microoperations are bit-wise operations, i.e., they work on the individual bits of data.
  - useful for bit manipulations on binary data .
  - useful for making logical decisions based on the bit value.
- There are,16 different logic functions that can be defined over two binary input variables

A	B	F_0	F_1	F_2	...	F_{13}	F_{14}	F_{15}
0	0	0	0	0	...	1	1	1
0	1	0	0	0	...	1	1	1
1	0	0	0	1	...	0	1	1
1	1	0	1	0	...	1	0	1

- However, most systems only implement four of these
  - AND ($\wedge$), OR ($\vee$), XOR ($\oplus$), Complement/NOT
- The others can be created from combination of these.

List of Logic Microoperations

- 16 different logic operations with 2 binary variables.

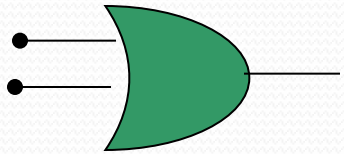
x y	0 0 1 1 0 1 0 1	*Boolean Function*	*Micro-Operations*	*Name*
	0 0 0 0	F0 = 0	F ← 0	Clear
	0 0 0 1	F1 = xy	F ← A ∧ B	AND
	0 0 1 0	F2 = xy'	F ← A ∧ B'	
	0 0 1 1	F3 = x	F ← A	Transfer A
	0 1 0 0	F4 = x'y	F ← A'∧ B	
	0 1 0 1	F5 = y	F ← B	Transfer B
	0 1 1 0	F6 = x ⊕ y	F ← A ⊕ B	Exclusive-OR
	0 1 1 1	F7 = x + y	F ← A ∨ B	OR
	1 0 0 0	F8 = (x + y)'	F ← (A ∨ B)'	NOR
	1 0 0 1	F9 = (x ⊕ y)'	F ← (A ⊕ B)'	Exclusive-NOR
	1 0 1 0	F10 = y'	F ← B'	Complement B
	1 0 1 1	F11 = x + y'	F ← A ∨ B	
	1 1 0 0	F12 = x'	F ← A'	Complement A
	1 1 0 1	F13 = x' + y	F ← A'∨ B	
	1 1 1 0	F14 = (xy)'	F ← (A ∧ B)'	NAND
	1 1 1 1	F15 = 1	F ← all 1's	Set to all 1's

Truth tables for 16 functions of 2 variables and the corresponding 16 logic micro-operations

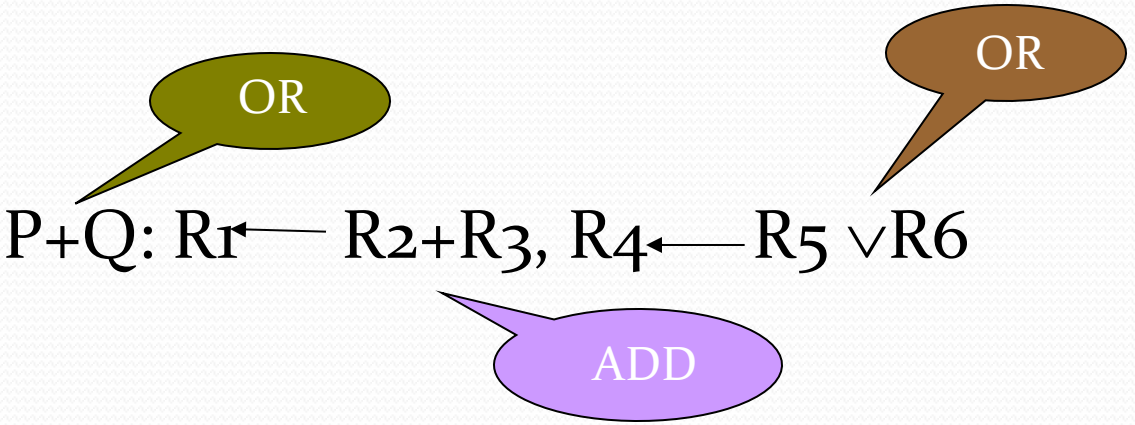
Logic Microoperations

OR Microoperation

- Symbol: $\vee$, +

Gate: 

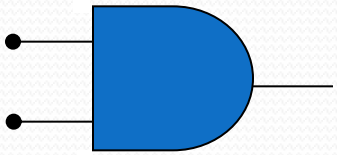
- Example: $100110_2 \vee 1010110_2 = 1110110_2$



Logic Microoperations

AND Microoperation

- Symbol: $\wedge$
- Gate:

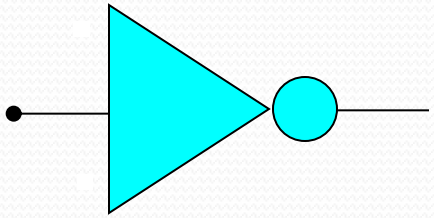


- Example: $100110_2 \wedge 1010110_2 = 0000110_2$

Logic Microoperations

Complement (NOT) Microoperation

- Symbol: $\overline{}$
- Gate:

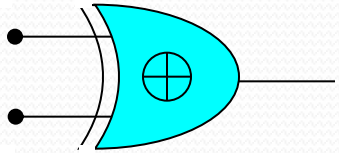


- Example: $\overline{1010110_2} = 0101001_2$

Logic Microoperations

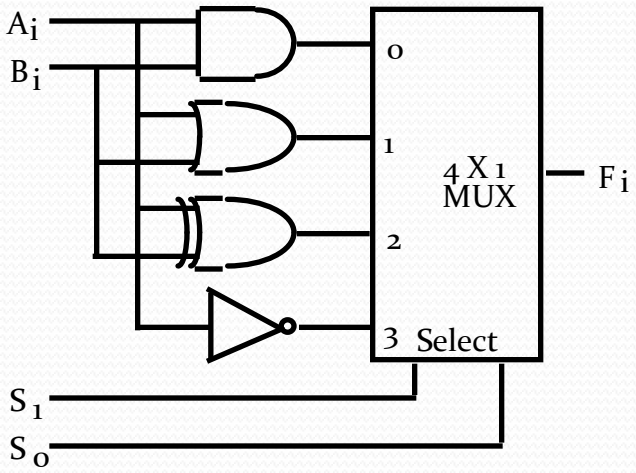
XOR (Exclusive-OR) Microoperation

- Symbol: $\oplus$
- Gate:



- Example: $100110_2 \oplus 1010110_2 = 1110000_2$

Hardware Implementation Of Logic Microoperations



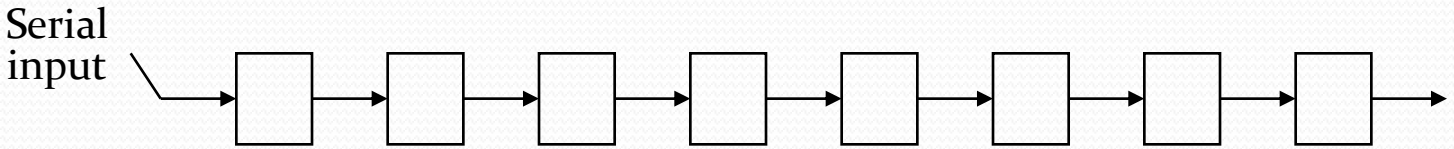
Function table

S_1 S_0	Output	μ-operation
0 0	$F = A \wedge B$	AND
0 1	$F = A \vee B$	OR
1 0	$F = A \oplus B$	XOR
1 1	$F = A'$	Complement

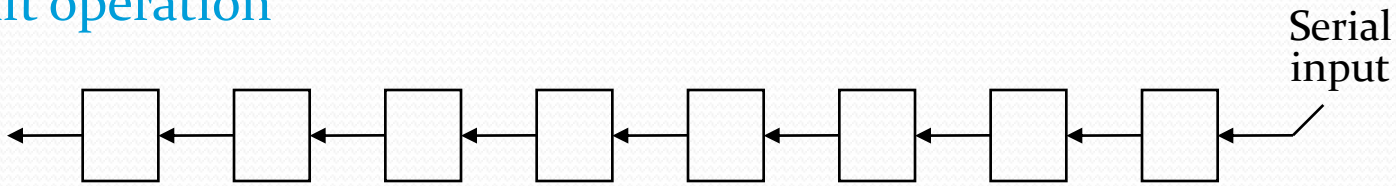
4. Shift Microoperations

- There are three types of shifts
  - *Logical shift*
  - *Circular shift*
  - *Arithmetic shift*
- What differentiates them is the information that goes into the serial input.

• A right shift operation

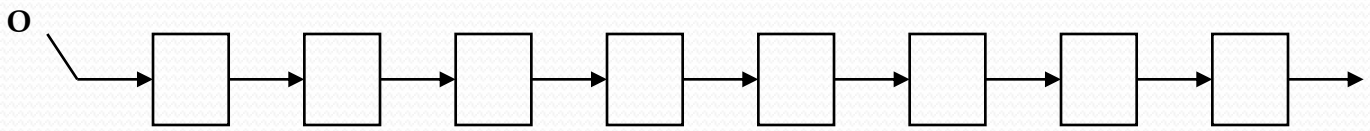


• A left shift operation

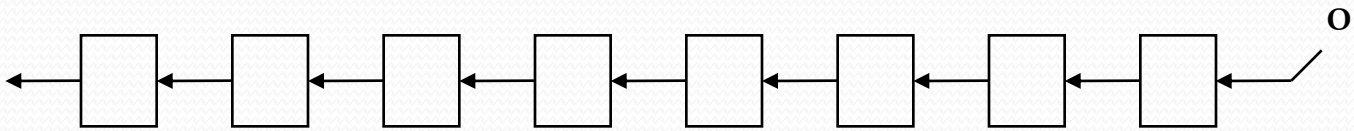


Logical Shift

- In a logical shift the serial input to the shift is a 0.
- A right logical shift operation:



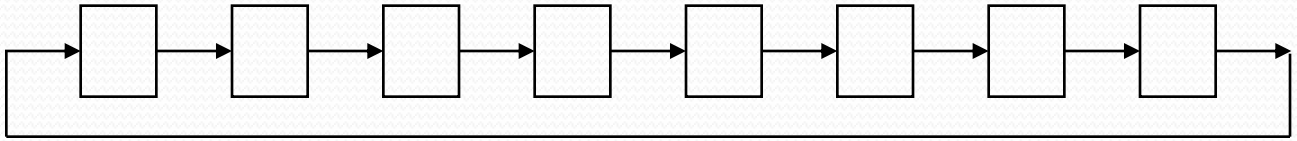
- A left logical shift operation:



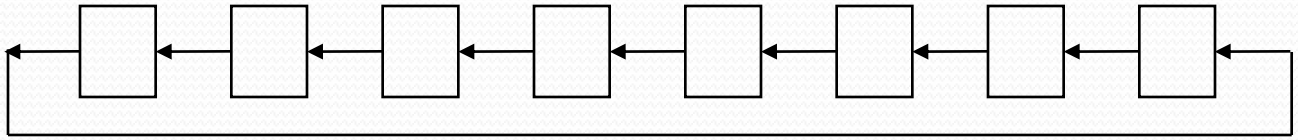
- In a Register Transfer Language, the following notation is used
  - *shl* for a logical shift left
  - *shr* for a logical shift right
  - Examples:
    - R2 ← *shr* R2
    - R3 ← *shl* R3

Circular Shift

- In a circular shift the serial input is the bit that is shifted out of the other end of the register.
- A right circular shift operation:



- A left circular shift operation:

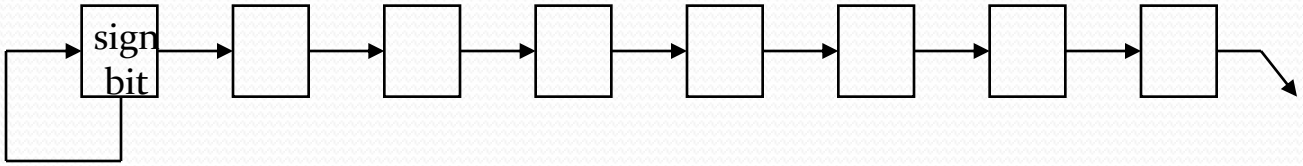


- In a RTL, the following notation is used
  - *cil* for a circular shift left
  - *cir* for a circular shift right
  - Examples:
    - R2 ← *cir* R2
    - R3 ← *cil* R3

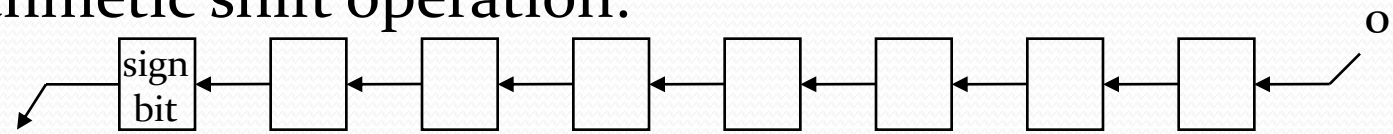
Arithmetic Shift

- An arithmetic shift is meant for signed binary numbers (integer).
- An arithmetic left shift multiplies a signed number by two.
- An arithmetic right shift divides a signed number by two.

- A right arithmetic shift operation:



- A left arithmetic shift operation:



Arithmetic Shift

- In a RTL, the following notation is used
  - *ashl* for an arithmetic shift left
  - *ashr* for an arithmetic shift right
  - Examples:
    - » R2 ← *ashr* R2
    - » R3 ← *ashl* R3

Arithmetic Shifts

- Shifts a signed binary number to the left or right
- An arithmetic shift-left multiplies a signed binary number by 2:

  ashl (00100): 01000

- An arithmetic shift-right divides the number by 2.

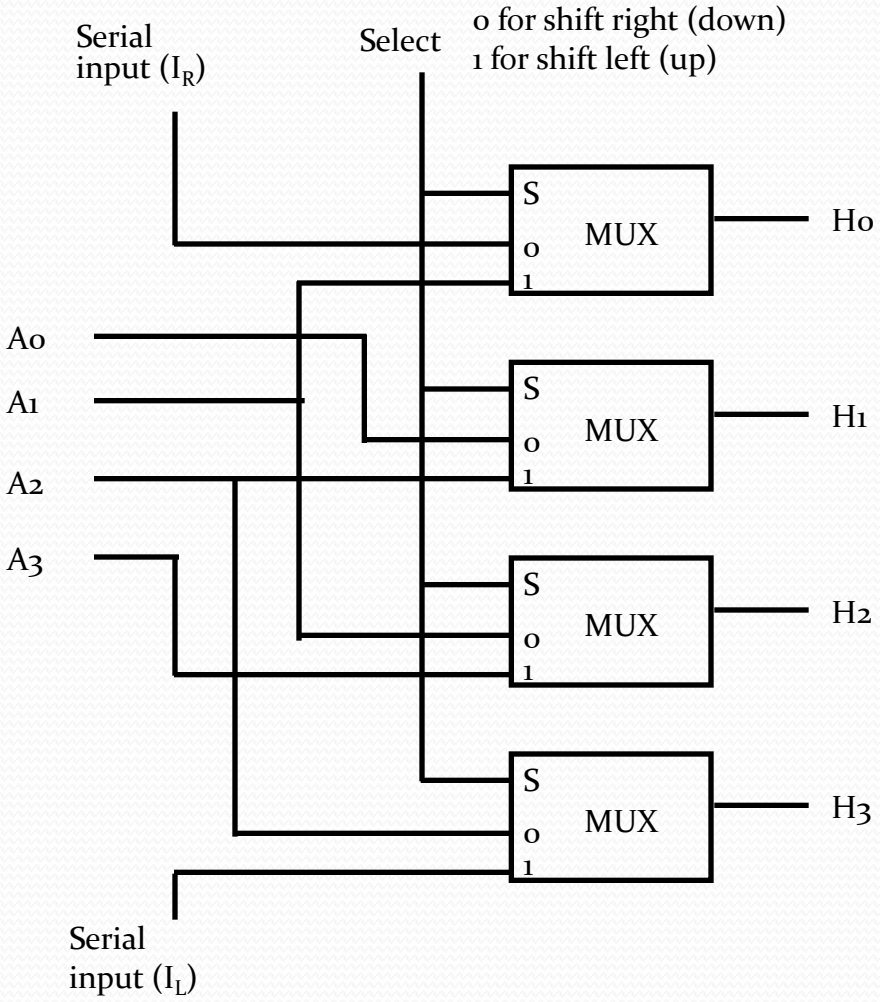
  ashr (00100) : 00010

- An overflow may occur in arithmetic shift-left, and occurs when the sign bit is changed (sign reversal).

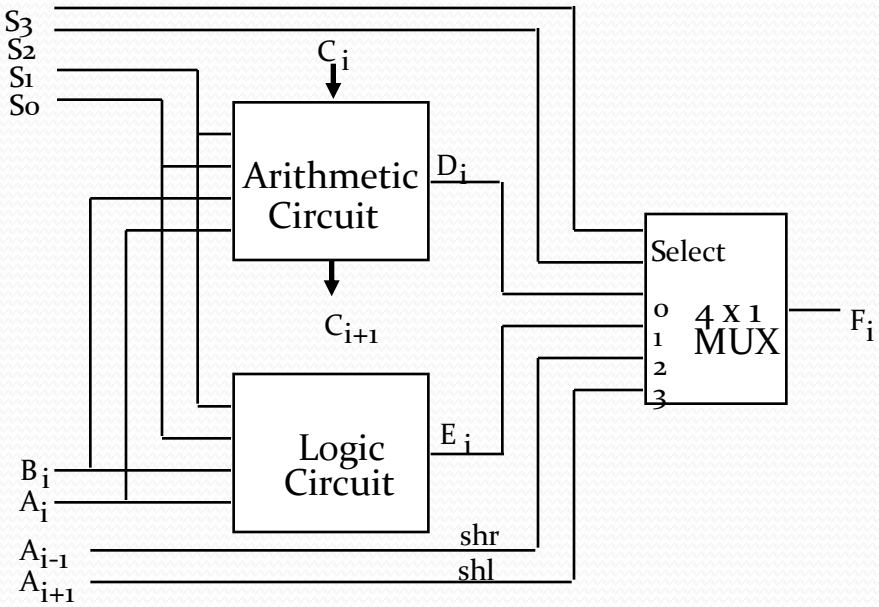
Shift Microoperations

- Example: Assume R1=11001110, then:
  - Arithmetic shift right once : R1 = 11100111
  - Arithmetic shift right twice : R1 = 11110011
  - Arithmetic shift left once : R1 = 10011100
  - Arithmetic shift left twice : R1 = 00111000
  - Logical shift right once : R1 = 01100111
  - Logical shift left once : R1 = 10011100
  - Circular shift right once : R1 = 01100111
  - Circular shift left once : R1 = 10011101

Hardware Implementation Of Shift Microoperations



5. Arithmetic Logic Shift Unit



S3	S2	S1	So	Cin	Operation	Function
0	0	0	0	0	F = A	Transfer A
0	0	0	0	1	F = A + 1	Increment A
0	0	0	1	0	F = A + B	Addition
0	0	0	1	1	F = A + B + 1	Add with carry
0	0	1	0	0	F = A + B'	Subtract with borrow
0	0	1	0	1	F = A + B'+ 1	Subtraction
0	0	1	1	0	F = A - 1	Decrement A
0	0	1	1	1	F = A	TransferA
0	1	0	0	X	F = A ∧ B	AND
0	1	0	1	X	F = A ∨ B	OR
0	1	1	0	X	F = A ⊕ B	XOR
0	1	1	1	X	F = A'	Complement A
1	0	X	X	X	F = shr A	Shift right A into F
1	1	X	X	X	F = shl A	Shift left A into F

6. Instruction Codes

- *Instruction Code*:

It is a group of bits that instruct the computer to perform a specific operation.

Instruction Format

Operation code :

The operation code of an instruction is a group of bits that define such as

add

subtract

multiply

shift

complement.

The Basic Computer

- The Basic Computer has two components, a processor and memory
- The memory has 4096 words in it
  - $4096 = 2^{12}$, so it takes 12 bits to select a word in memory
- Each word is 16 bits long

Instruction Format

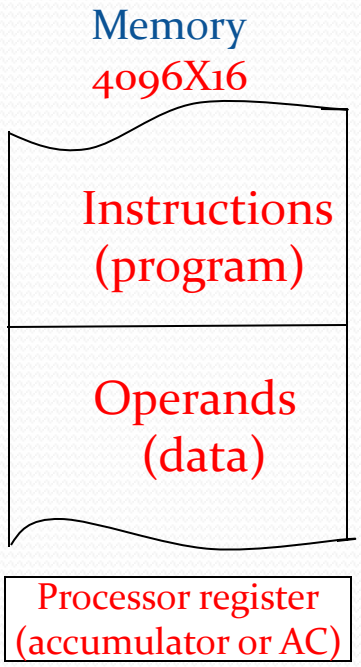
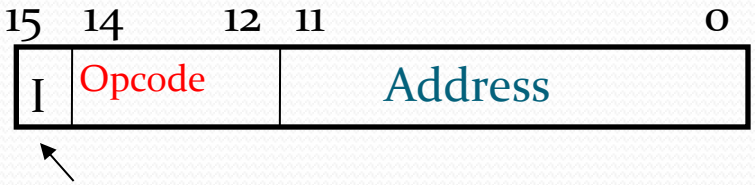


Fig: Stored program organization

Instruction Format

- A computer instruction is often divided into two parts
  - An *opcode* (Operation Code) that specifies the operation for that instruction
  - An *address* that specifies the registers and/or locations in memory to use for that operation
- In the Basic Computer, since the memory contains 4096 (= 2^{12}) words, we needs 12 bit to specify which memory address this instruction will use



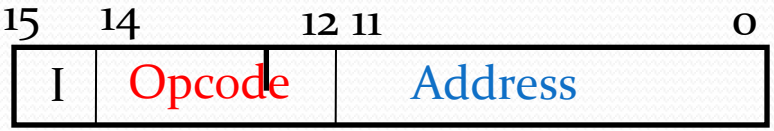
Instruction Format

- In the Basic Computer, bit 15 of the instruction specifies the *addressing mode* (0: direct addressing, 1: indirect addressing)
- Since the memory words, and hence the instructions, are 16 bits long, that leaves 3 bits for the instruction's opcode.

7. Basic Computer Instructions

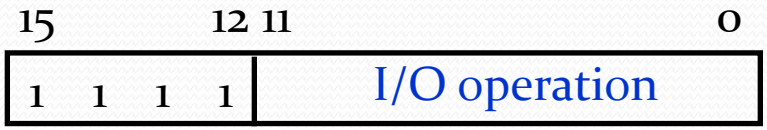
- Basic Computer Instruction Format

Memory-Reference Instructions (OP-code = 000 ~ 110)



Register-Reference Instructions (OP-code = 111, I = 0)

Input-Output Instructions (OP-code =111, I = 1)



Basic Computer Instructions

Symbol	Hex Code I = 0	Hex Code I = 1	Description
AND	0xxx	8xxx	AND memory word to AC
ADD	1xxx	9xxx	Add memory word to AC
LDA	2xxx	Axxx	Load AC from memory
STA	3xxx	Bxxx	Store content of AC into memory
BUN	4xxx	Cxxx	Branch unconditionally
BSA	5xxx	Dxxx	Branch and save return address
ISZ	6xxx	Exxx	Increment and skip if zero
CLA	7800		Clear AC
CLE	7400		Clear E
CMA	7200		Complement AC
CME	7100		Complement E
CIR	7080		Circulate right AC and E
CIL	7040		Circulate left AC and E
INC	7020		Increment AC
SPA	7010		Skip next instr. if AC is positive
SNA	7008		Skip next instr. if AC is negative
SZA	7004		Skip next instr. if AC is zero
SZE	7002		Skip next instr. if E is zero
HLT	7001		Halt computer
INP	F800		Input character to AC
OUT	F400		Output character from AC
SKI	F200		Skip on input flag
SKO	F100		Skip on output flag
ION	F080		Interrupt on
IOF	F040		Interrupt off

Instruction Set Completeness

A computer should have a set of instructions so that the user can construct machine language programs to evaluate any function that is known to be computable.

Instruction Types

Functional Instructions

- Arithmetic, logic, and shift instructions
- ADD, CMA, INC, CIR, CIL, AND, CLA

Transfer Instructions

- Data transfers between the main memory and the processor registers
- LDA, STA

Control Instructions

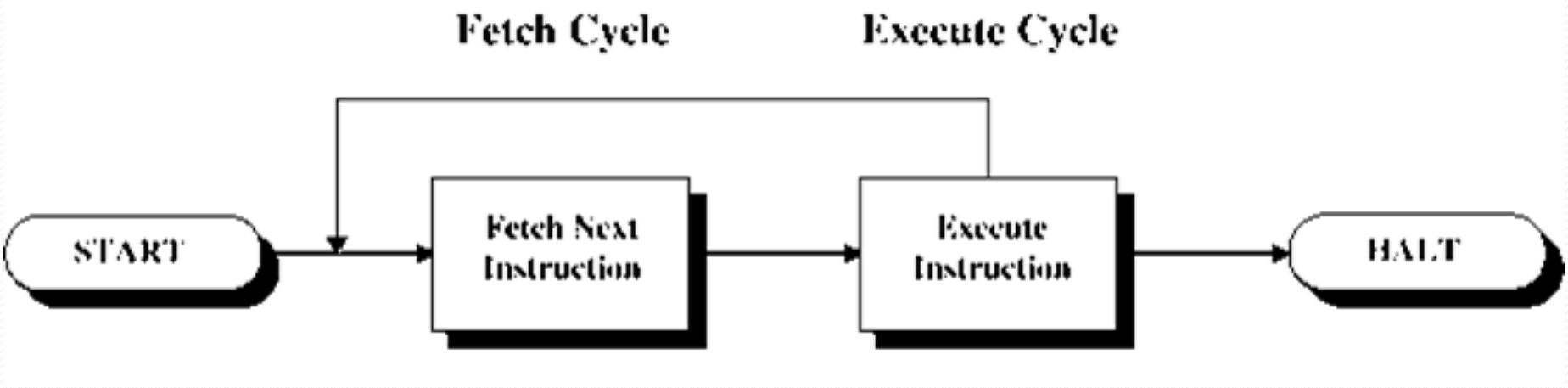
- Program sequencing and control
- BUN, BSA, ISZ

Input/Output Instructions

- Input and output
- INP, OUT

8. Instruction Cycle

- In Basic Computer, a machine instruction is executed in the following cycle:
  1. Fetch an instruction from memory
  2. Decode the instruction
  3. Read the effective address from memory if the instruction has an indirect address
  4. Execute the instruction

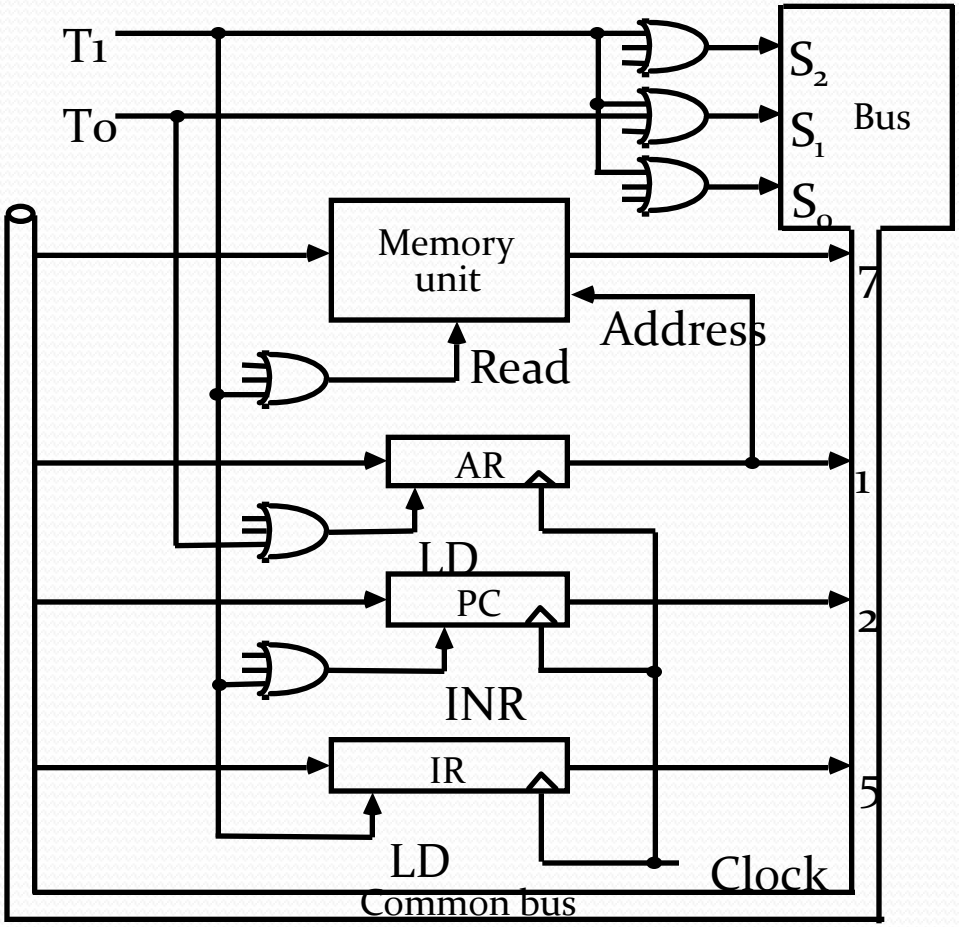


- After an instruction is executed, the cycle starts again at step 1, for the next instruction.
- *Note*: Every different processor has its own (different) instruction cycle .

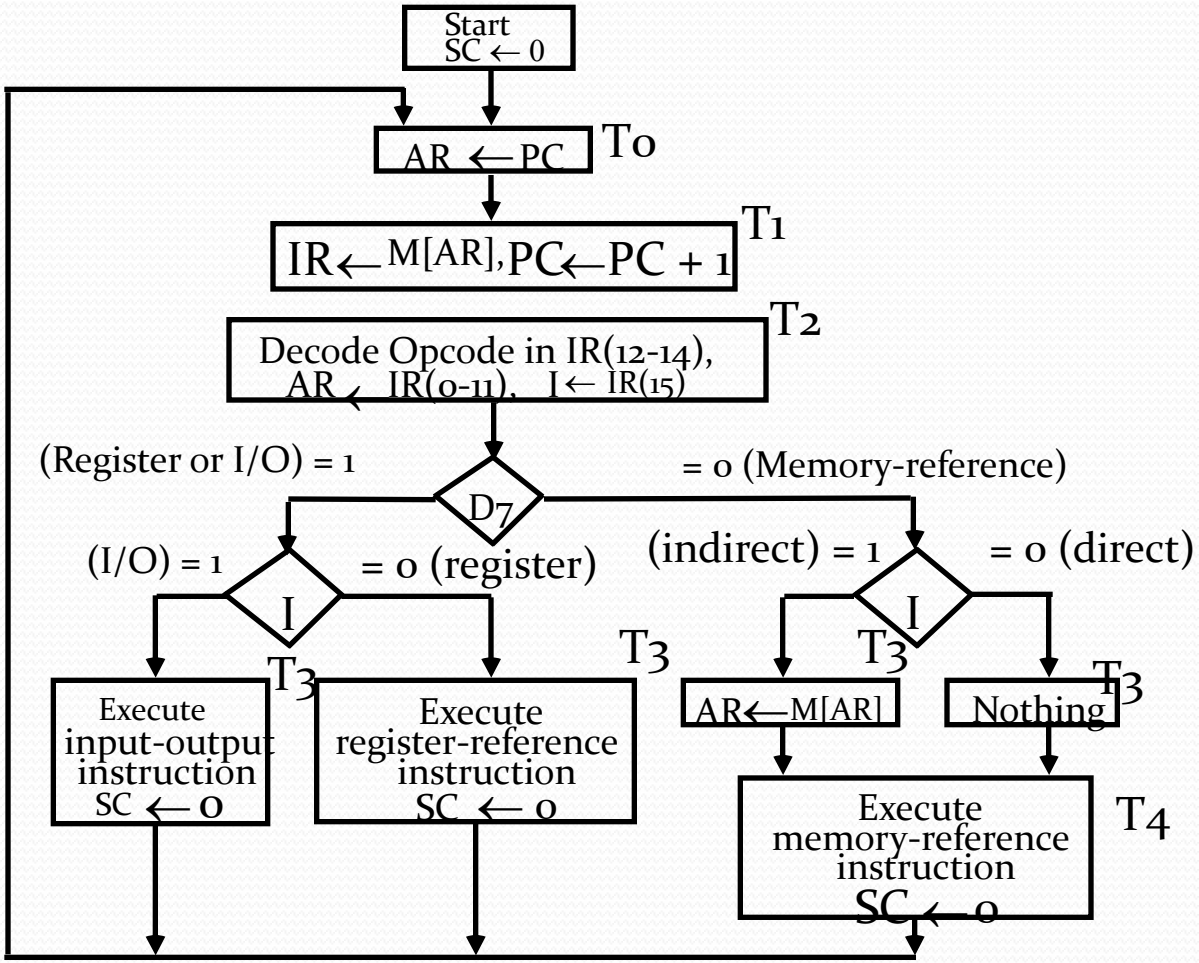
Fetch and Decode

- Fetch and Decode

To: AR ← PC ($S_0S_1S_2$=010, To=1)
T1: IR ← M [AR], PC ← PC + 1 (S0S1S2=111, T1=1)
T2: D0, . . . , D7 ← Decode IR(12-14), AR ← IR(0-11), I ← IR(15)



Determine The Type Of Instruction



D'_7IT_3: AR ← M[AR]
$D'_7I'T_3$: Nothing
$D_7I'T_3$: Execute a register-reference instr.
D_7IT_3: Execute an input-output instr.

Register Reference Instructions

Register Reference Instructions are identified when

- $D_7 = 1$, $I = 0$
- Register Ref. Instr. is specified in $b_0 \sim b_{11}$ of IR
- Execution starts with timing signal T_3

$r = D_7\, I'\, T_3$ => Register Reference Instruction

$B_i = IR(i)$, i=0,1,2,...,11

	r:	SC ← 0
CLA	rB_{11}:	AC ← 0
CLE	rB_{10}:	E ← 0
CMA	rB_9:	AC ← AC'
CME	rB_8:	E ← E'
CIR	rB_7:	AC ← shr AC, AC(15) ← E, E ← AC(0)
CIL	rB_6:	AC ← shl AC, AC(0) ← E, E ← AC(15)
INC	rB_5:	AC ← AC + 1
SPA	rB_4:	if (AC(15) = 0) then (PC ← PC+1)
SNA	rB_3:	if (AC(15) = 1) then (PC ← PC+1)
SZA	rB_2:	if (AC = 0) then (PC ← PC+1)
SZE	rB_1:	if (E = 0) then (PC ← PC+1)
HLT	rB_0:	S ← 0 (S is a start-stop flip-flop)

9. Memory Reference Instructions

Symbol	Operation Decoder	Symbolic Description
AND	D_0	$AC \leftarrow AC \wedge M[AR]$
ADD	D_1	$AC \leftarrow AC + M[AR], E \leftarrow C_{out}$
LDA	D_2	$AC \leftarrow M[AR]$
STA	D_3	$M[AR] \leftarrow AC$
BUN	D_4	$PC \leftarrow AR$
BSA	D_5	$M[AR] \leftarrow PC, PC \leftarrow AR + 1$
ISZ	D_6	$M[AR] \leftarrow M[AR] + 1$, if $M[AR] + 1 = 0$ then $PC \leftarrow PC+1$

- The effective address of the instruction is in AR and was placed there during timing signal T_2 when I = 0, or during timing signal T_3 when I = 1
- Memory cycle is assumed to be short enough to complete in a CPU cycle
- The execution of MR instruction starts with T_4

AND to AC

D_0T_4: $DR \leftarrow M[AR]$ Read operand

D_0T_5: $AC \leftarrow AC \wedge DR, SC \leftarrow 0$ AND with AC

ADD to AC

D_1T_4: $DR \leftarrow M[AR]$ Read operand

D_1T_5: $AC \leftarrow AC + DR, E \leftarrow C_{out}, SC \leftarrow 0$ Add to AC and store carry in E

Memory Reference Instructions

LDA: Load to AC

D_2T_4: DR ← M[AR]

D_2T_5: AC ← DR, SC ← 0

STA: Store AC

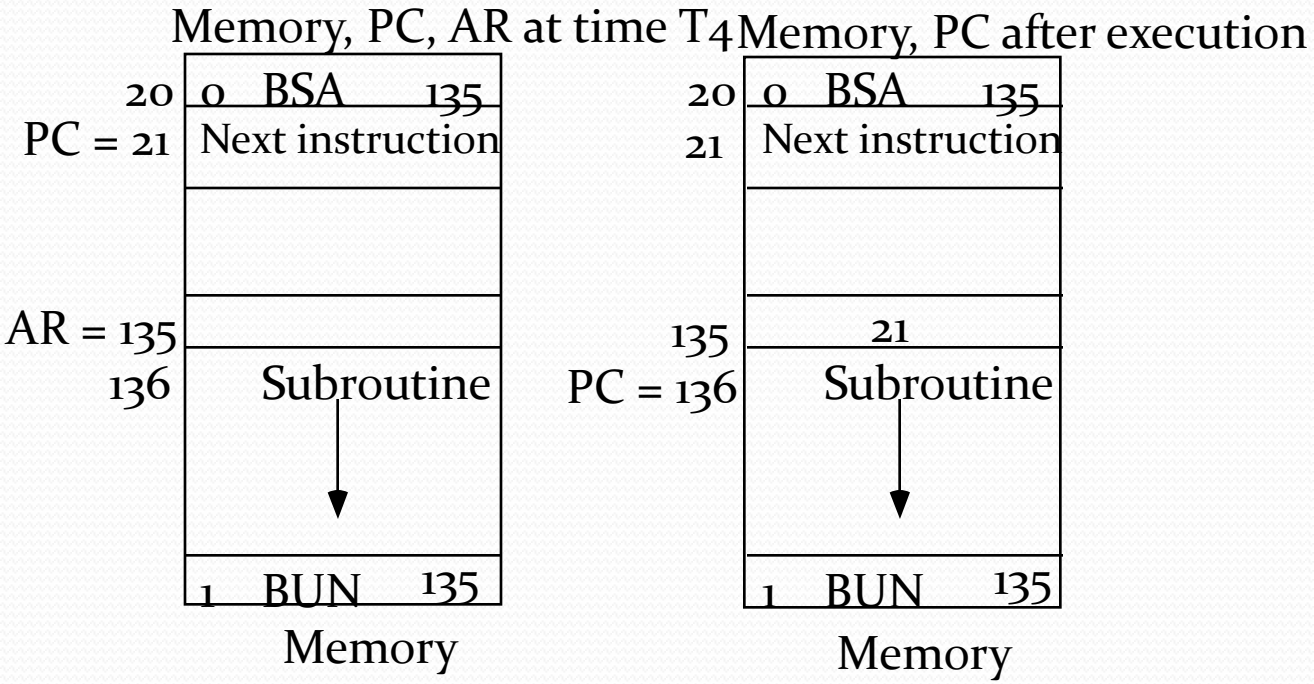
D_3T_4: M[AR] ← AC, SC ← 0

BUN: Branch Unconditionally

D_4T_4: PC ← AR, SC ← 0

BSA: Branch and Save Return Address

M[AR] ← PC, PC ← AR + 1



Memory Reference Instructions

BSA:

D_5T_4: $M[AR] \leftarrow PC,\ AR \leftarrow AR + 1$

D_5T_5: $PC \leftarrow AR,\ SC \leftarrow 0$

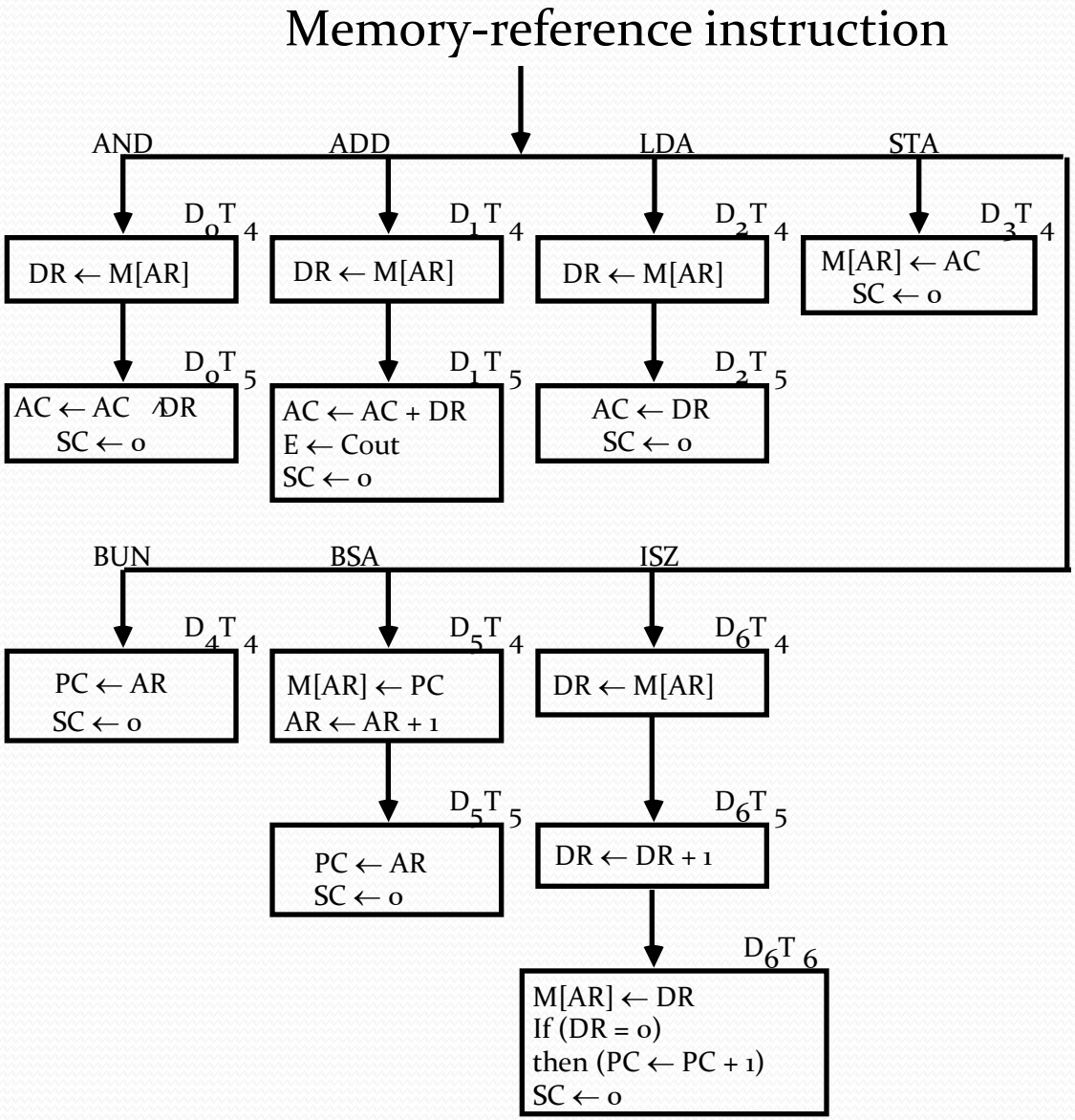
ISZ: Increment and Skip-if-Zero

D_6T_4: $DR \leftarrow M[AR]$

D_6T_5: $DR \leftarrow DR + 1$

D_6T_4: $M[AR] \leftarrow DR,\ \text{if } (DR = 0) \text{ then } (PC \leftarrow PC + 1),\ SC \leftarrow 0$

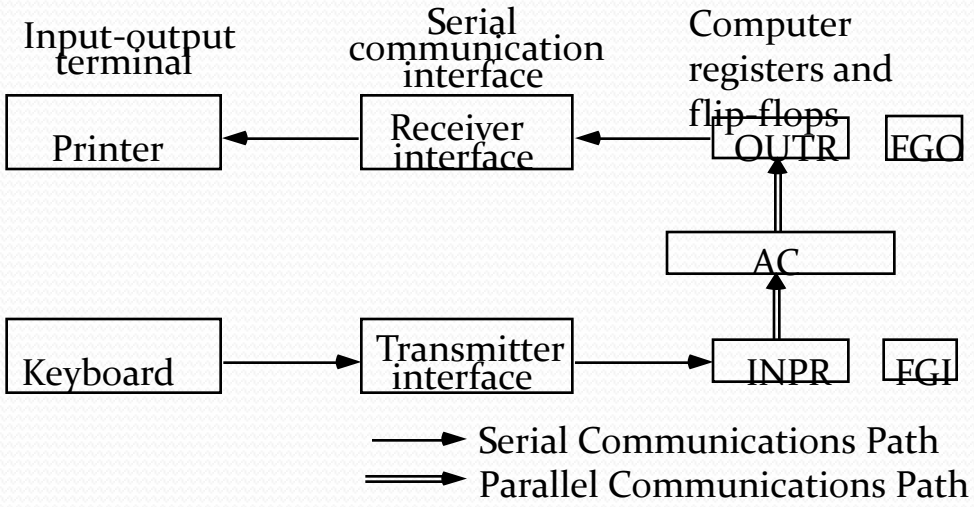
Flowchart For Memory Reference Instructions



10. Input-Output And Interrupt

A Terminal with a keyboard and a Printer

- Input-Output Configuration



INPR	Input register - 8 bits
OUTR	Output register - 8 bits
FGI	Input flag - 1 bit
FGO	Output flag - 1 bit
IEN	Interrupt enable - 1 bit

- The terminal sends and receives serial information
- The serial info. from the keyboard is shifted into INPR
- The serial info. for the printer is stored in the OUTR
- INPR and OUTR communicate with the terminal serially and with the AC in parallel.
- The flags are needed to *synchronize* the timing difference between I/O device and the computer

Input-Output Instructions

$D_7IT_3 = p$
$IR(i) = B_i$, i = 6, ..., 11

	p:	SC ← 0	Clear SC
INP	pB_{11}:	AC(0-7) ← INPR, FGI ← 0	Input char. to AC
OUT	pB_{10}:	OUTR ← AC(0-7), FGO ← 0	Output char. from AC
SKI	pB_9:	if(FGI = 1) then (PC ← PC + 1)	Skip on input flag
SKO	pB_8:	if(FGO = 1) then (PC ← PC + 1)	Skip on output flag
ION	pB_7:	IEN ← 1	Interrupt enable on
IOF	pB_6:	IEN ← 0	Interrupt enable off

Program-Controlled Input/Output

Program-controlled I/O

- Continuous CPU involvement

  I/O takes valuable CPU time

- CPU slowed down to I/O speed

- Simple

- Least hardware

Interrupt Initiated Input/Output

-Open communication only when some data has to be passed --> *interrupt*.

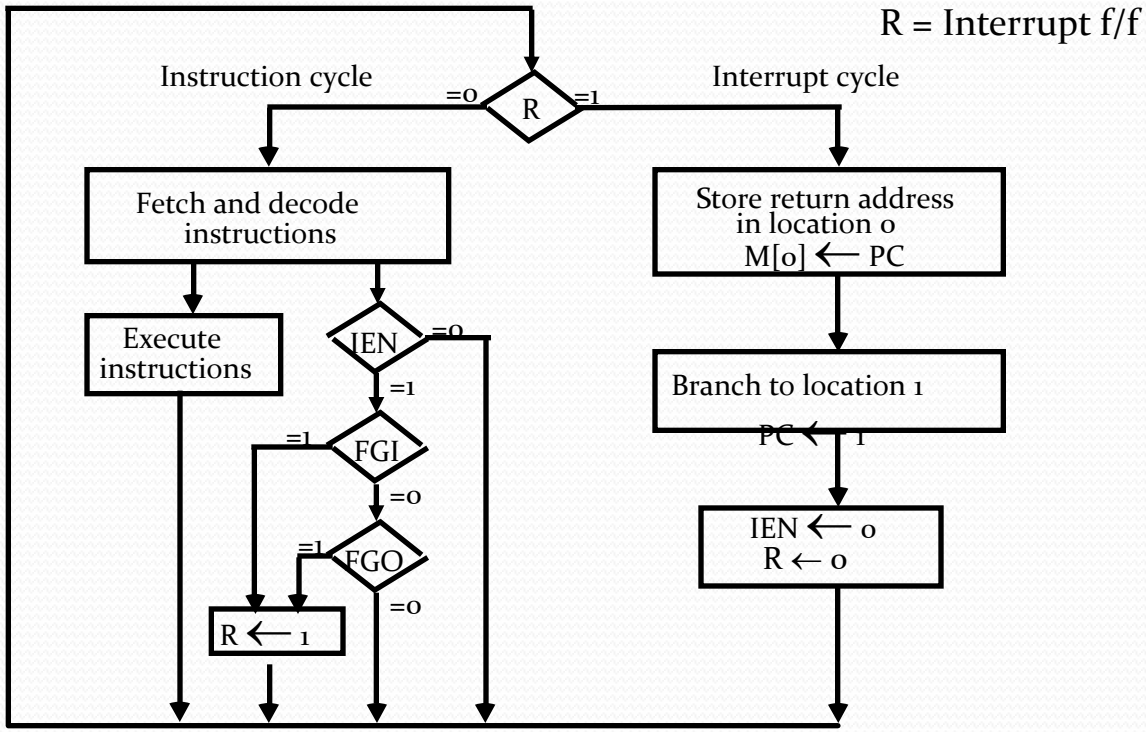
-- The I/O interface, instead of the CPU, monitors the I/O device.

- When the interface founds that the I/O device is ready for data transfer, it generates an interrupt request to the CPU
- Upon detecting an interrupt, the CPU stops momentarily the task it is doing, branches to the service routine to process the data transfer, and then returns to the task it was performing.

* IEN (Interrupt-enable flip-flop)

  - can be set and cleared by instructions.
  - when cleared, the computer cannot be interrupted.

Flowchart For Interrupt Cycle



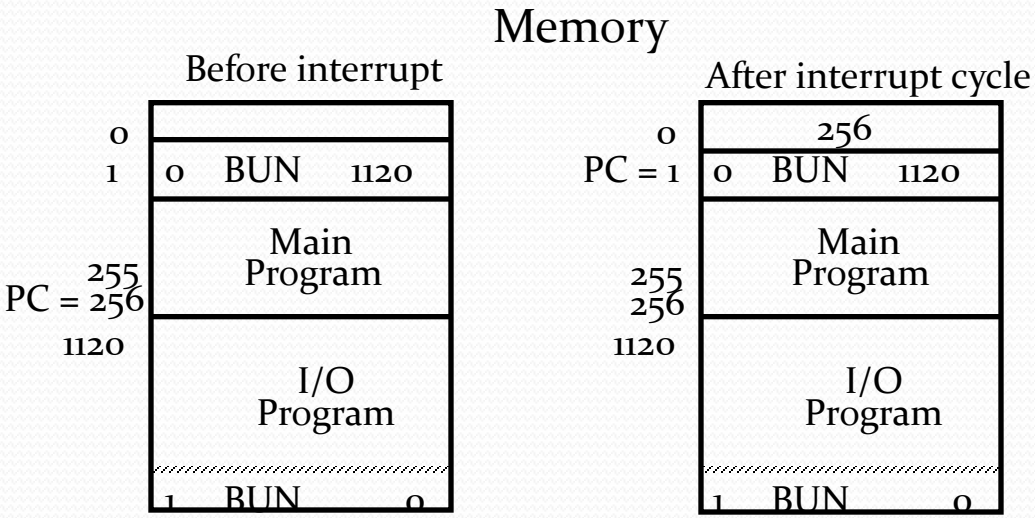
- The interrupt cycle is a HW implementation of a branch and save return address operation.

- At the beginning of the next instruction cycle, the instruction that is read from memory is in address 1.

- At memory address 1, the programmer must store a branch instruction that sends the control to an interrupt service routine.

- The instruction that returns the control to the original program is "indirect BUN 0".

Register Transfer Operations In Interrupt Cycle



Register Transfer Statements for Interrupt Cycle

- R F/F $\leftarrow$ 1 if IEN (FGI + FGO)$T_0'T_1'T_2'$

$\Leftrightarrow T_0'T_1'T_2'$ (IEN)(FGI + FGO): R $\leftarrow$ 1

- The fetch and decode phases of the instruction cycle

must be modified ➔Replace T_0, T_1, T_2 with $R'T_0$, $R'T_1$, $R'T_2$

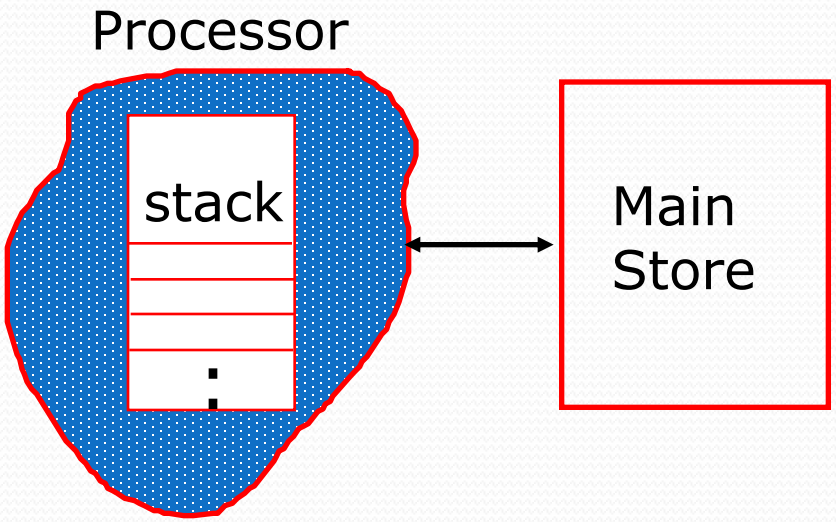
- The interrupt cycle :

RT_0: AR $\leftarrow$ 0, TR $\leftarrow$ PC

RT_1: M[AR] $\leftarrow$ TR, PC $\leftarrow$ 0

RT_2: PC $\leftarrow$ PC + 1, IEN $\leftarrow$ 0, R $\leftarrow$ 0, SC $\leftarrow$ 0

11. A Stack Machine

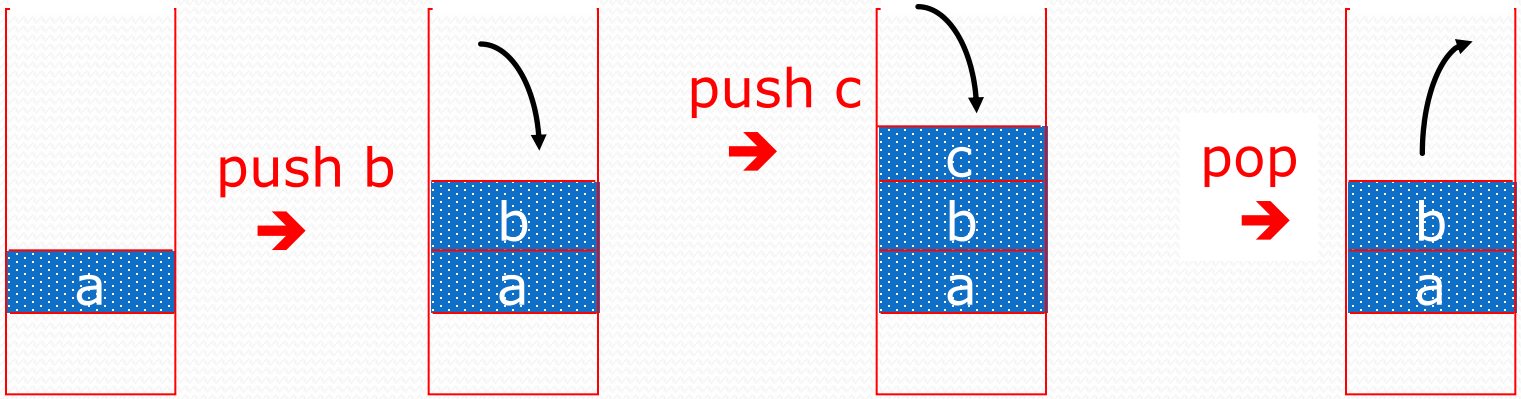


A Stack machine has a stack as a part of the processor state

typical operations:

push, pop, +, *, ...

Instructions like + implicitly specify the top 2 elements of the stack as operands.

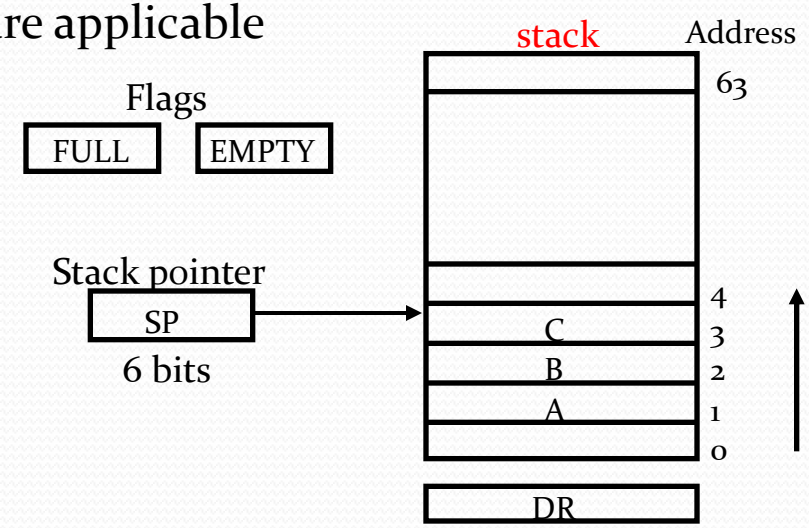


Register Stack Organization

Stack

- Very useful feature for nested subroutines, nested interrupt services
- Also efficient for arithmetic expression evaluation
- Storage which can be accessed in LIFO
- Pointer: SP
- Only PUSH and POP operations are applicable

Register Stack



Push, Pop operations

PUSH

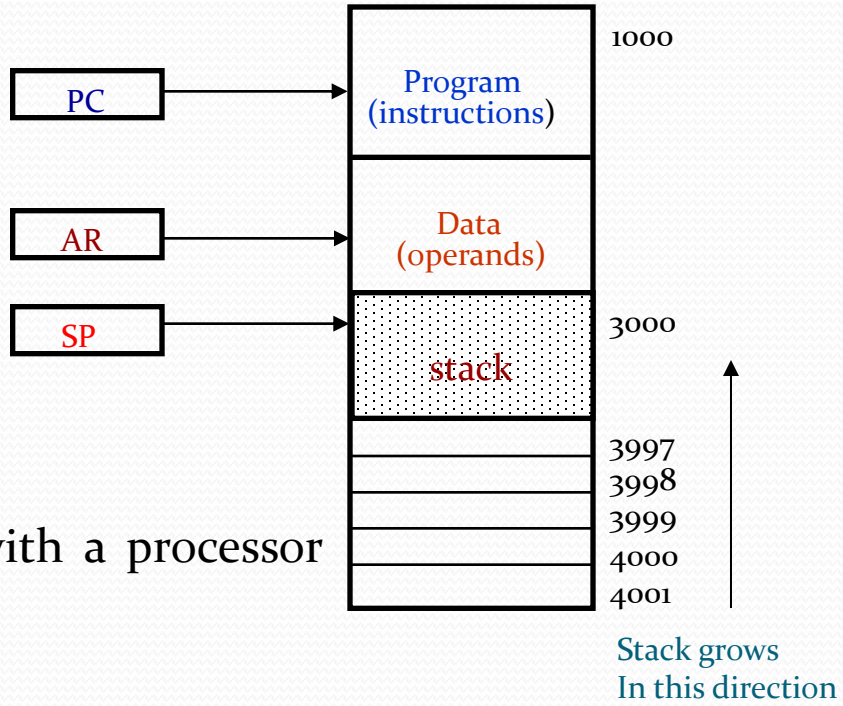
SP ← SP + 1
M[SP] ← DR
If (SP = 0) then (FULL ← 1)
EMPTY ← 0

POP

DR ← M[SP]
SP ← SP – 1
If (SP = 0) then (EMPTY ← 1)
FULL ← 0

Memory Stack Organization

Memory with Program, Data, and Stack Segments



- A portion of memory is used as a stack with a processor register as a stack pointer.

- PUSH: $SP \leftarrow SP - 1$

  $M[SP] \leftarrow DR$

- POP: $DR \leftarrow M[SP]$

  $SP \leftarrow SP + 1$

- Most computers do not provide hardware to check stack overflow (full stack) or underflow (empty stack) → must be done in software

Reverse Polish Notation

- Arithmetic Expressions: A + B

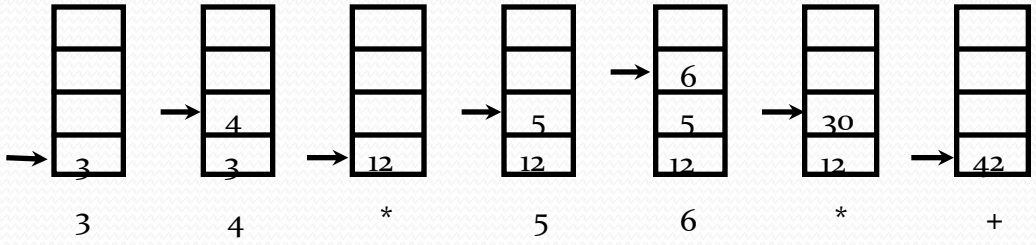
A + B	Infix notation
+ A B	Prefix or Polish notation
A B +	Postfix or reverse Polish notation

- The reverse Polish notation is very suitable for stack manipulation

- Evaluation of Arithmetic Expressions

Any arithmetic expression can be expressed in parenthesis-free Polish notation, including reverse Polish notation

$(3 * 4) + (5 * 6) \Rightarrow 3\ 4\ *\ 5\ 6\ *\ +$



Processor Organization

- In general, most processors are organized in one of 3 ways
  - Single register (Accumulator) organization
    - Basic Computer is a good example.
    - Accumulator is the only general purpose register.
  - General register organization
    - Used by most modern computer processors.
    - Any of the registers can be used as the source or destination for computer operations.
  - Stack organization
    - All operations are done using the hardware stack.
    - For example, an OR instruction will pop the two top elements from the stack, do a logical OR on them, and push the result on the stack.

12. Instruction Format

Instruction Fields

OP-code field - specifies the operation to be performed

Address field - designates memory address(es) or a processor register(s)

Mode field - determines how the address field is to be interpreted (to get effective address or the operand)

- The number of address fields in the instruction format depends on the internal organization of CPU.
- The three most common CPU organizations:

Single accumulator organization:

```
ADD     X              /* AC ← AC + M[X]  */
```

General register organization:

```
ADD     R1, R2, R3       /* R1 ← R2 + R3  */
ADD     R1, R2           /* R1 ← R1 + R2  */
MOV     R1, R2           /* R1 ← R2  */
ADD     R1, X            /* R1 ← R1 + M[X]  */
```

Stack organization:

```
PUSH    X                /* TOS ← M[X]  */
ADD
```

Three, and Two-Address Instructions

- Three-Address Instructions

Program to evaluate X = (A + B) * (C + D) :

```
ADD   R1, A, B   /*  R1 ← M[A] + M[B]          */
ADD   R2, C, D          /*  R2 ← M[C] + M[D]          */
MUL   X, R1, R2         /*  M[X] ← R1 * R2            */
```

- Results in short programs
- Instruction becomes long (many bits)

- Two-Address Instructions

Program to evaluate X = (A + B) * (C + D) :

```
MOV   R1, A          /* R1 ← M[A]          */
ADD   R1, B          /* R1 ← R1 + M[A]  */
MOV   R2, C          /* R2 ← M[C]          */
ADD   R2, D          /* R2 ← R2 + M[D]  */
MUL   R1, R2         /* R1 ← R1 * R2      */
MOV   X, R1          /* M[X] ← R1          */
```

One and Zero-Address Instructions

- One-Address Instructions
  - Use an implied AC register for all data manipulation
  - Program to evaluate X = (A + B) * (C + D) :

```
LOAD    A    /*  AC ← M[A]          */
ADD     B    /*  AC ← AC + M[B]     */
STORE   T    /*  M[T] ← AC          */
LOAD    C    /*  AC ← M[C]          */
ADD     D    /*  AC ← AC + M[D]     */
MUL     T    /*  AC ← AC * M[T]     */
STORE   X    /*  M[X] ← AC          */
```

- Zero-Address Instructions
  - Can be found in a stack-organized computer
  - Program to evaluate X = (A + B) * (C + D) :

```
PUSH    A    /*  TOS ← A                    */
PUSH    B    /*  TOS ← B                    */
ADD          /*  TOS ← (A + B)              */
PUSH    C    /*  TOS ← C                    */
PUSH    D    /*  TOS ← D                    */
ADD          /*  TOS ← (C + D)              */
MUL          /*  TOS ← (C + D) * (A + B)    */
POP     X    /*  M[X] ← TOS                 */
```

The End